



UNIVALI

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia - EMCT
Ciência da Computação

REPOSITORY PATTERN

Gustavo Henrique Stahl Müller, Paulo Henrique Rohling
e Vinícius Gabriel Martins.



Sumário

- Definição
- Problematização
- Solução
- Exemplos
- Vantagens e Desvantagens

Definição



O padrão de projeto **Repository** tem como objetivo criar uma abstração para a persistência de dados, normalmente com uma interface de comunicação entre a aplicação e o banco de dados, auxiliando o isolamento da camada de acesso a dados (DAL) e aplicando assim o princípio da persistência ignorante (PI), de modo que a camada de negócio não fique sensível à uma troca de SGBD.

Problematização

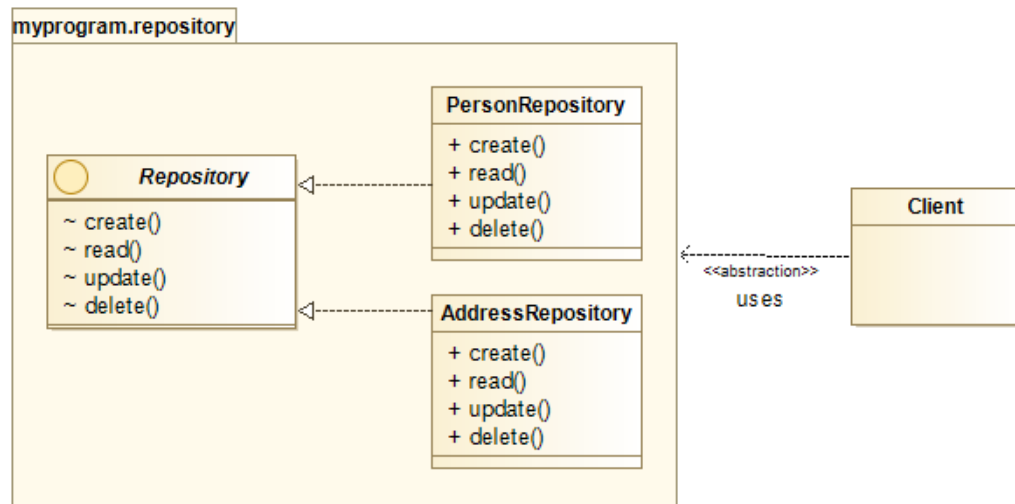


Imagine um sistema em que o acesso ao banco de dados não é centralizado, ou seja, não há uma **divisão de responsabilidade** para cuidar desta tarefa, então toda classe que necessitar estabelecer a comunicação e manipulá-lo deve implementar tal comunicação.

O que acontece caso a complexidade do sistema aumente ou se for preciso migrar, por algum motivo, do MySQL para SQL Server? Em quantos lugares a refatoração seria necessária? **Muitos.**

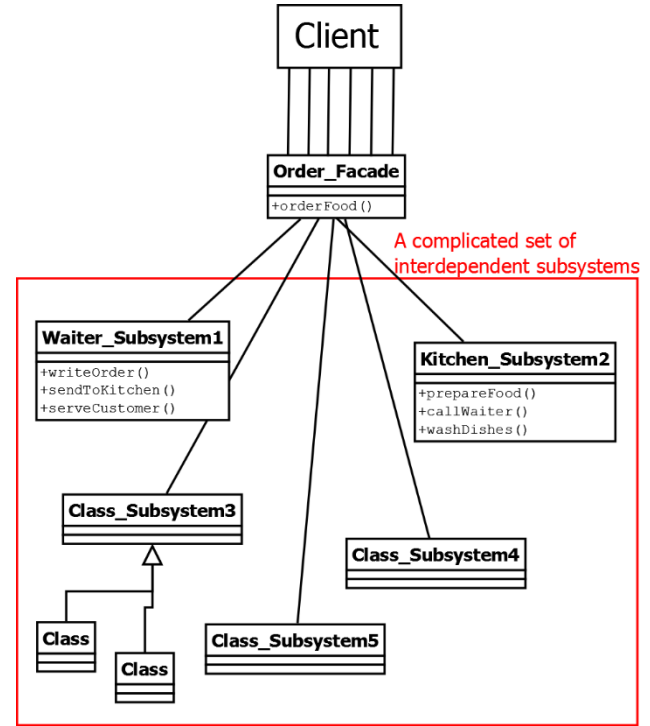
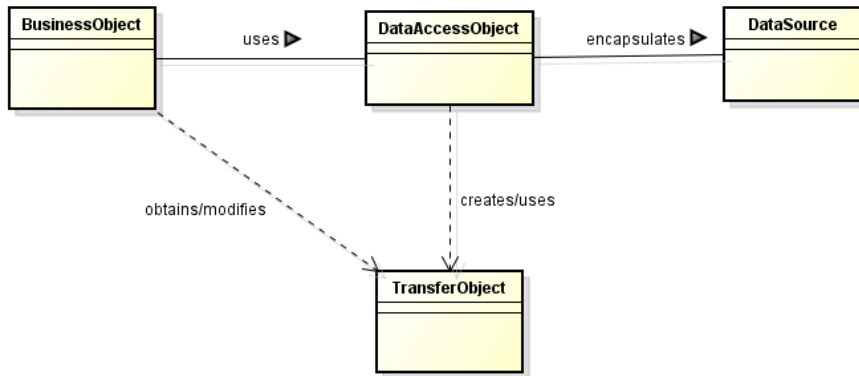
Solução

- Usar o padrão **Repository**;
- Acesso aos dados é centralizado em um local;
- Acrescenta **flexibilidade**.



Semelhança com outros patterns

- Semelhante ao DAO e Façade;
- DAO não está ligado à lógica do negócio;
- Façade foca na simplicidade.



Exemplo

```
package example.model;

public class Client {
    private int id;
    private String name;
    private String cpf;

    public Client(int id, String name, String cpf) {}

    public int getId() {}

    public void setId(int id) {}

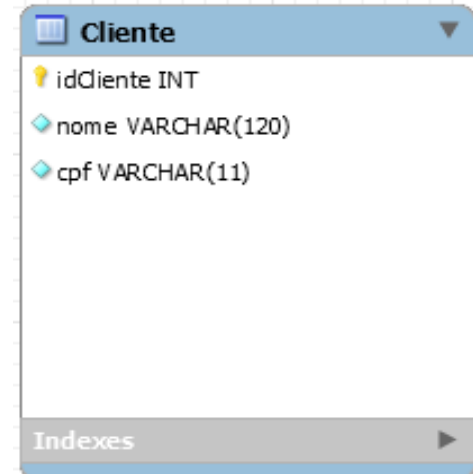
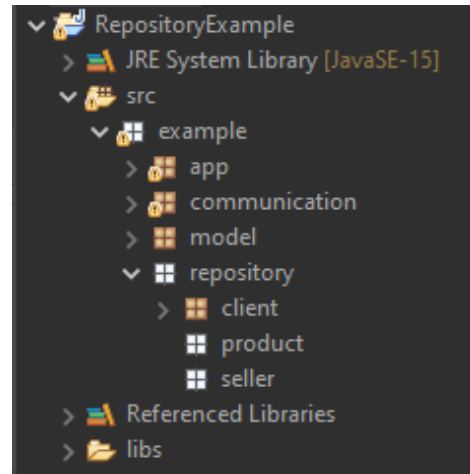
    public String getName() {}

    public void setName(String name) {}

    public String getCpf() {}

    public void setCpf(String cpf) {}

    public String toString() {}
}
```



Exemplo

```
1 package example.communication;
2
3 import java.sql.Connection;
4
5
6
7 public class ConnectionFactory {
8     public static Connection getConnection() {
9         return getMySQLConnection();
10    }
11
12    public static Connection getMySQLConnection() {
13        String url = "jdbc:mysql://localhost:3306/repository_test";
14        String user = "root";
15        String pw = "1234";
16
17        try {
18            return DriverManager.getConnection(url, user, pw);
19        } catch (SQLException e) {
20            System.out.println(e.getMessage());
21            throw new RuntimeException(e);
22        }
23    }
24
25    public static Connection getPostgreSQLConnection() {
26        String url = "jdbc:postgresql://localhost:5432/repository_test";
27        String user = "postgres";
28        String pw = "1234";
29
30        try {
31            return DriverManager.getConnection(url, user, pw);
32        } catch (SQLException e) {
33            System.out.println(e.getMessage());
34            throw new RuntimeException(e);
35        }
36    }
37 }
38
```


Exemplo

```
1 package example.repository.client;
2
3 import java.util.List;
4
5
6
7 public interface ClientRepositoryInterface {
8     Client getClient(int id);
9     List<Client> getAllClients();
10    void insertClient(Client client);
11    void updateClient(Client client);
12    void deleteClient(int id);
13 }
```

```
1 package example.repository.client;
2
3 import example.communication.ConnectionFactory;
4
5
6
7 public class ClientRepository implements ClientRepositoryInterface {
8
9     @Override
10    public Client getClient(int id) {
11        String query = "SELECT IDCLIENTE, NOME, CPF FROM CLIENTE WHERE IDCLIENTE = ?";
12
13        Connection cn = ConnectionFactory.getConnection();
14        PreparedStatement statement;
15        Client targetClient = null;
16
17        try {
18            statement = cn.prepareStatement(query);
19            statement.setInt(1, id);
20            ResultSet result = statement.executeQuery();
21            result.next();
22            targetClient = new Client(result.getInt("idcliente"),
23                                    result.getString("nome"),
24                                    result.getString("cpf"));
25
26            cn.close();
27        } catch (SQLException e) {
28            System.out.println(e.getMessage());
29        }
30
31        return targetClient;
32    }
33
34    public List<Client> getAllClients() {}
35
36    public void insertClient(Client client) {}
37
38    public void updateClient(Client client) {}
39
40    public void deleteClient(int id) {}
41 }
```

Exemplo

```
1 package example.app;
2
3 import java.util.List;
4
5
6
7 public class RepositoryTest {
8     public static void main(String[] args) {
9         System.out.println("INÍCIO");
10
11         ClientRepository rep = new ClientRepository();
12
13         Client c1 = new Client(0, "José da Fonseca", "12345678945");
14         rep.insertClient(c1);
15
16         Client c2 = rep.getClient(6);
17         c2.setName("José da Fonseca Silva");
18
19         rep.updateClient(c2);
20
21         List<Client> clients = rep.getAllClients();
22         for(Client c : clients) {
23             System.out.println(c);
24         }
25     }
26 }
```

Exemplo



INÍCIO

```
Client [id=1, name=Paulo Henrique, cpf=000000000000]
```

...

```
Client [id=6, name=José da Fonseca Silva, cpf=12345678945]
```

```
Client [id=7, name=José da Fonseca, cpf=12345678945]
```

Exemplo

```
1 package example.communication;
2
3 import java.sql.Connection;
4
5
6
7 public class ConnectionFactory {
8     public static Connection getConnection() {
9         return getMySQLConnection();
10    }
11
12    public static Connection getMySQLConnection() {
13        String url = "jdbc:mysql://localhost:3306/repository_test";
14        String user = "root";
15        String pw = "1234";
16
17        try {
18            return DriverManager.getConnection(url, user, pw);
19        } catch (SQLException e) {
20            System.out.println(e.getMessage());
21            throw new RuntimeException(e);
22        }
23    }
24
25    public static Connection getPostgreSQLConnection() {
26        String url = "jdbc:postgresql://localhost:5432/repository_test";
27        String user = "postgres";
28        String pw = "1234";
29
30        try {
31            return DriverManager.getConnection(url, user, pw);
32        } catch (SQLException e) {
33            System.out.println(e.getMessage());
34            throw new RuntimeException(e);
35        }
36    }
37 }
38
```

Vantagens e Desvantagens

- Baixo Acoplamento entre as Classes;
- Um **Repository** para cada entidade;
- Flexibilidade de Código;
- Facilidade na Manipulação de Dados;
- Fácil de Testar;
- **Mal compreendido;**
- **Um **Repository** para cada entidade;**

Referências



- **Entendendo o Repository Pattern.** Disponível em: <<https://medium.com/@renicius.pagotto/entendendo-o-repository-pattern-fcdd0c36b63b>>
- **Façade.** Disponível em: <<https://pt.wikipedia.org/wiki/Fa%C3%A7ade>>
- **Introduction to Repository Design Pattern.** Disponível em: <<https://cubettech.com/resources/blog/introduction-to-repository-design-pattern>>
- **Objeto de acesso a dados.** Disponível em: <https://pt.wikipedia.org/wiki/Objeto_de_acesso_a_dados>
- **Qual a diferença entre DAO e Repository?** Disponível em: <<https://pt.stackoverflow.com/questions/12927/qual-a-diferen%C3%A7a-entre-dao-e-repository/>>
- **Repository Pattern.** Disponível em: <<https://deviq.com/design-patterns/repository-pattern>>
- **Repository Pattern, Done Right.** Disponível em: <<https://www.codeproject.com/Articles/526874/Repository-pattern-done-right>>
- **The Repository Pattern.** Disponível em: <https://shawnmc.cool/2015-01-08_the-repository-pattern>
- **Você entende Repository Pattern?** Disponível em: <<https://medium.com/laraveltips/voc%C3%AA-entende-repository-pattern-voc%C3%AA-est%C3%A1-certo-disso-d739ecaf544e>>